ActInf ModelStream 018.1: Robust Decision-Making Via Free Energy Minim

ModelStream | 1:26:23

Hello, welcome everyone. It is April 28th, 2025, and we're in ACTIMF Model Stream 18.1 on robust decision-making via free energy minimization. So thank you to the authors who have joined today, looking forward to this presentation and walkthrough. So thanks again, and to you all, go for it. Okay, thank you, Daniel, and thank you for the invite, for inviting us here. So we are, let's say, that's three of us today. Arash, unfortunately, was the other first author of the paper that made most of the mathematics. We cannot be here today, but we will basically go through the paper that we have just submitted. So let me start with thanking all the co-authors. So these are the co-authors of the paper that is currently on the archive. The paper provides a computational model, which we call Dr. Free, which relies basically on free energy minimization for policy computation and tries to install robustness into decision-making. So our angle is to design policies, making decisions that are robust against certain ambiguities. Our background, so from the group in Salerno, so Arash, Josef, and myself, is that of control theory. So I apologize if sometimes maybe I would speak about some control terminology. So I will start with some motivation. First of all, motivating the problem, why we should be able to design agents, autonomous agents that can make decisions amid ambiguity. And we will, let's say, check some differences between what is probably natural intelligence, how natural intelligence agents behave, and autonomous agents, current autonomous agents. Then I will jump into our solution for free energy minimization, which is Dr. Free, this computational model I'm going to talk about. And then I will let the floor to Josefa for some discussion about the code to finally conclude with some remarks. very well. So let's start to set out, set out to the ground first. So it is undeniable that autonomous agents have achieved groundbreaking performance. For example, in the figure here in the slide on the left, we can design today agents embodied in robots, for example, that can make very complex tasks, like building towers from blocks as the popular child game. But if we look closely to the picture, the agent is performing these tasks in a very controlled environment, which is a lab. And perhaps the policies that the agent learns, so the policies, the method that the agent uses to make actions are quite inflexible to changes against the environment and against the task. In contrast, natural intelligence, so for example, a child that is playing exactly the same game can do the same task in a much more robust way. And this is, of course, only a toy problem. Imagine what would happen if we had these mismatches with the environment, between the training and the environment. And what would happen if the agent starts to misbehave? We could have failures in the robot, for example, that could cause damages to the environment, to the robot and maybe to people around them. So the angle that we are going to to discuss today is an approach based on the free energy minimization that tries to install robustness into the decision making problem. Again, I speak here from the viewpoint of a person that comes from control theory. So we would like really to design policies, so making decisions that allow agents to compute optimal actions that are also robust against certain mismatches that we will see soon.

I know this is a much broader topic because there is a lot of research that could feed a lot of research programs here that spans from, I don't know, science is designing better actuators, designing better sensors, better software, etc. Here we are really interested in the intelligent step. So we can see how we can do this. And in fact, this is a challenge that requires interdisciplinary foundations. And the approach that we present is in fact interdisciplinary. It mixes a few key ingredients, of course, free energy minimization that will serve us as a framework to cast policy computation as an optimal control problem. So with optimal control, I mean the discipline, the problem of finding optimal actions that minimize a given cost function that formalizes the agent task. And the way we will frame the problem, we will see that this will have a lot to do with optimization, in particular optimization in the space of densities, in the probability space, which is technically an infinite dimensional optimization problem. So the mix of these three key ingredients gives us our computational model, our free energy computational model, which in fact consists of an infinite dimensional free energy formulation and a resolution engine that computes the optimal policy, essentially, and is able to output the optimal policy with provable guarantees.

What is the gain of this approach? First of all,

the gain of this approach is the gain of this approach. So, for example, it can provide a normative framework to

equip natural agents to explain behaviors of natural agents. Then we could try to reverse engineer, if we could distill this framework into a robot, for example, as we're doing experiments, we would be able to equip artificial agents with robust free energy minimization properties. And of course, coming from a rather theoretical field like control, it can provide some elegant mathematical methods.

So the account that we use, the unifying account that we use for co-policy computation is free energy. Now, free energy is very broad. There is lots of literature, it really comes, pops up everywhere in several apparently different domains, coming from, of course, neuroscience, as a theory to explain self-organization, brain functions and behaviors in general, to technology, some technological fields such as optimization, mirror descent, maximum entropy, statistical learning, etc. So this is really a unifying account that we are going to leverage for autonomous decision making.

So let's start to formalize these decision making problems. So we are going to have an agent, which will embody our model, that is a free energy minimizing agent, and the agent interacts with an environment. Now the agent is equipped with a model of the world, a world model, which is obtained, for example, from training. So this trained model, we call it PBAR, is basically any model of the world that the agent has available. Now it can come from training, it can come from past experience, it can come from any sort of different pieces of information. What is important is that this model is available offline to the agent, and will ground, and the agent will ground its decisions based on this model.

Now, the environment, the world, is, as we know, non-linear, stochastic, and non-stationary in general. So it is a challenge world, the one we live in. And it provides to the agent some information. This information, we call it

state x. This x is a label for both observations and beliefs that are relevant to the agent task. So

xk minus one here is the information that is available to the agent in order to make a decision. They can be observations and can be belief functions. Based on this information, the agent, well, it has available the classical generative model, and it has subject preferences that we express as cost functions. And based on this, as we will see today, we will be computing a policy, which is a randomized probability function, basically a probability density function, from which the agent computes an action u. Okay, so u k is going to be the set of actions that are determined by the agent. And these actions are obtained by sampling from the available information. Now, what is the point? The point is that the environment, the world, the world, is different by definition from the world model. Okay, so we would like to study what happens when the environment trained model differs from the real environment. Just in terms of notation, as I said, I'll try to keep the mathematics at the minimum. I just like to flag that random capital letters means random variables. Lowercase letters are just realizations. And subscripts are time dependent. Whenever you see bold, it is basically a vector. So we have this framework here. So we have this setup. We have an agent that is interacting with the world around it. We have essentially a feedback system where information in the forms of state feeds the agent, the agent computes an action, and the actions pushes the world to move in a new state. So this is a back and forth information that we can capture mathematically. Of course, there is an initial condition, there is an initial state for the environment, which is a prior, px0 here. Then what happens? The agent, given this initial information, using the cost function, the preferences and the generative model, will determine an action. In our framework, as I just illustrated, the action is obtained by sampling from a policy, p uk given xk minus one. This is the policy that the agent will use to generate uk given the current information. And this uk, once sampled, is sent to the environment, to the world, and it will make it transition to a new state xk. Now, this is just a picture, a snapshot at one time step. This thing happens, this thing happens continuously between zero and let's say some final time, n. And we can capture all of these as a product. So we have a product of these probability density functions or probability mass functions, depending if we are working with continuous or discrete states. All in all, this big product of probability mass or density functions is just a joint, a big joint probability density or a big joint probability mass function that in MATLAB style notation we indicate with p0, n. So this closed loop probability function, p0, n captures the behavior, the closed loop behavior of the agent. So it means that captures the interactions between the agent and the world. Now we could do technically exactly the same thing with the generative model. So we could make a big product of all the conditional densities, given some initial prior, and we can call this probability function q0, n. So this q0, n is the joint probability density function of states and actions from the generative model.

So the subject of our model, the goal of our model is that of computing the method to generate the action. So it's to compute the policies. And the policies are going to be computed via free energy minimization. So how do we cast the free energy minimization problem? First of all, we use the definition of variational free energy. Okay, so variational free energy is essentially a complexity plus an expected cost and expected loss terms. Now complexity is statistical complexity. It is measured using the standard Kuhlbeck-Leibler divergence. So the Kuhlbeck-Leibler divergence,

I'm sure everybody that does active inference is familiar with, but it has basically this expression here. So it is not a distance because it is not symmetric, but it is non-negative and it is zero if and only if p and q are the same. So we have this complexity term that quantifies how far the closed loop system p0, n is from the generative model. So it is a sort of an error term. And then there is an expected loss. Well, the expected loss is just the expectation over the closed loop behavior, p0, n, of the summation between the state and the action costs. The good news is that we know how to solve these control problems when basically the trained model is the same as the world model, where basically you see in green here in the slide we have pk being the trained model and being exactly the same as the world model.

So the good news is that we know how to solve this, uh, this optimization or optimal for problems in a number of

very recent papers that we got, that we got published. Uh, and as a matter of fact, we know how to compute policies in this setup.

Let's consider one example that we will use throughout this live stream here, which leverages a very nice platform from Georgia Tech, which is called the Robotarium. The Robotarium is a robotic platform that allows you to deploy

experiments remotely. So these guys from Georgia Tech basically provide you robots and provide you a small

workspace where you can deploy your algorithms and they run, and they run the experiments for you.

So the Robotarium uses, uh, uh, uh, a, uh, a rover, a, uh, unicycle robot where xk, the state, is the position of the robot,

and uk are the input speeds at the wheels. Now in the experiments that are also in the playprint that is attached to this

live stream, uh, the robot has to move in an environment that is filled with obstacles. So whatever you see here in red, uh,

uh, are obstacles and the green position is the goal destination of the robot. So of course we would like to solve a navigation task where we reach the, um, the green position while avoiding the reds.

Now let's use, let's minimize the free energy pretending that the world model and the robot model are exactly the same. But let's assume, let's say that we learn a model of the robot from a set of corrupted data. Let's say so that there is a very small difference between the real model of the robot,

which is the one in the bottom in blue, and the, the, the, the model obtained from training, which is the one in red. So you see that the, the difference is only in one parameter, uh, and the difference in that parameter is about 10%. So as an engineer speaking now, 10% difference in a, in a parameter is not that much. So it's something that the system should be able to tolerate.

Now let's consider, let's set up the, the, the, the policy computation framework. Let's say we have a very simple generative model, uh, where the, the state distribution is an ocean centered in the goal destination and Q, the policy distribution, the generative policy distribution, is just a uniform distribution. And let's say that the preferences, the cost of the agent, uh, formalize collision avoidance with the obstacles. So our cost functions are basically something like the, the one that you see

in the bottom of the picture. So we have walls, mountains that emulate the presence of the obstacles, and then you have a minimum in the bottom left area, bottom left part of the work area, minus one, minus 1.5. Okay. So let's see what happens if we pretend, actually, if we don't know that the real robot is different from the robot that we learned during training. So let's apply the results that are known in the literature. So we just minimize this cost function on here in the, in the policy space.
And what we see is, uh, uh, something that is quite, uh, uh, quite relevant for us. What we see is that in this very simple setup, by using the optimal policy and by only having a 10% discrepancy between the real model and the model from training, basically the robot fails in the task in, uh, basically every time except then in trivial situations. By trivial situations, I mean situations where the shortest path between the initial position of the robot and the final destination is obstacle-free. You see that only in the red, uh, uh, in the, in the red curve there is the only initial position that the robot is able to achieve the final destination. All the others, whenever there is an obstacle along the path, so that the robot should change direction to avoid obstacles, the, uh, the, the robot fails, does not achieve the destination. Of course, this means if it was a real robot interacting in a real environment, this means that there is a crash with the wall or with the environment in general, and that can be even dangerous. So, this is exactly the type of behavior that we would like to avoid. So, first of all, um, this mismatch between the real robot and the robot model available during training is called ambiguity. So, we have an ambiguous model and this ambiguity can have catastrophic effects. So, it can lead to failures that can be catastrophic for the agent and for the surroundings. So, in the next part of the presentation and then on the concept, I will follow up with the code with implementations, we will discuss how our model can mitigate this, uh, this issue. So, in particular, our model relies on free energy minimization, but we will see that basically we will minimize the worst case free energy. And that means that implies a free energy maximization step, which is quite unusual. Then the model will allow us to establish what are the best performance that can be achieved by an agent that follows the model. Plus, we will have some interpretability, um, interpretability property. So, the pro-, the pro-, the policy can be, uh, the cost driving the actions of an agent can be inferred from the viewpoint of an external observer.
And also clarifies some cognitive behaviors, like, this hinting, hint, tries to explain some cognitive behaviors in the, in the, in the regimes of large and small ambivities.
So, this is what the model will do. Now, let's see what the model actually is. And, uh, we should start by formalizing a little bit the problem, and we should check why the robot was failing. We should understand why the robot was failing in the previous example, when the ambiguity was not considered. So, let's go back to our picture. In the end, what happened before was that the real robot was different from the robot learned during training. So, the two PDFs there were different, and this small mismatch created the crashes that we saw in the picture just a few minutes before. We can make a step, an abstraction step, and realize that we can look at these objects in the space of density. Okay, now, these two objects are just two points in the space of density. Let's say, P bar is the training model, the nominal model that the agent has available, and P is going to be a different point in this space.
We are interested in protecting, protecting the decisions of the agent against all possible

perturbation in this space that are in a suitably, suitably defined set. Okay, so what we are going to define is a neighborhood, a sort of neighborhood, although it's not properly mathematically neighborhood, which is centered around the nominal model. And we are going to call this radius eta, which we call a radius of ambiguity. So, the set inside the circle here is going to be called ambiguity set, and it is going to be the radius of ambiguity.

We can formalize things a little bit more. There are some technicalities that I refer to the interested readers to the paper, and we can say that basically this set, this circle is the set of all probability densities that have Kulbeck-Laber divergence from the nominal model of at most eta. Okay, again, eta is an ambiguity radius, quantifies how ambiguous the model, the training model is, can be state dependent, time dependent, and action dependent, of course. And it has a straightforward interpretation. Basically, if eta is small for a given action and state pair, this means that the agent is quite confident or believes it is confident in the model P-bar, in the training model. Otherwise, it means that it has low confidence in the model that it is available.

So, we have formalized this set. Oh, sorry, this car maybe wanted to say something. I saw a hand raising. You did, and I do want to say something.

And I should apologize because I have to go in a few minutes, but I just wanted the opportunity to congratulate you on this work before I go away.

I think a number of things that this work brings to the table, and the first thing I think you've really highlighted is the intimate relationship between the sort of biomimetic applications of the free energy principle and active inference and control theoretic approaches, which of course really start to matter when in real world implementations and neuro-robotics and the like.

So, I just wanted to acknowledge that that's a really important point of convergence.

And just also foreground the key advance that you've framed and you're about...

I'm going to miss my favorite part because you're about to explain my favorite part, but just to sort of put it in context, you know, to my mind, this brings a very fresh and different view of ambiguity to the table from the point of view of active inference, because unlike active inference in discrete state space models where ambiguity is all about an intrinsic part of expected free energy, ambiguity in this instance I think is much closer to the notion that you'd find in economics, where it's not only uncertainty about what will happen if I do this, but it's I don't even know the game I'm playing. And this is really, literally, a deep aspect of uncertainty. And basically, I think what you're bringing to the table here is a principled way of resolving that or incorporating that uncertainty, incorporating that uncertainty quantification into a vanilla free energy minimizing process.

So, to my mind, you know, this is very much and literally a deep solution.

I, you know, one can look at it in many ways, one can look at this as basically a clever way of Bayesian model averaging by including uncertainty about the very structure, about the very parameterization of the model itself by bringing hyperpriors to the table that say, look, you know, I know what to do if this is the right model, but I have this extra kind of ambiguity, this extra kind of uncertainty that actually inherits from not knowing what the model in and of itself is, and you've provided a particular structure to that.

I just want to conclude by, and this is something that you might pick up in later discussion, but it also, this sort of kind of robust Bayesian model averaging or handling ambiguity, not only accommodates the uncertainty about the very structure of the model itself, but it also resolves something which is generic to variational approaches, which is the overconfidence problem. So even if you knew the right model, the mean field approximations that are inherent in variational bays and, you know, the minimization of variational free energy inevitably lead to an overconfidence, and in principle, I think the technology you're about to do a deep dive on would also resolve that overconfidence problem, which has been long-standing, you know, in variational inference, and of course really starts to bite when you actually put robots into play. So I'll stop there, but just to thank you for involving me in this work, and again, congratulate you.

Thank you Carl for the kind words, and we can follow up on the many future and interesting directions that come from this work, that start from this work.

Thank you Carl.

Okay, so I'm gonna continue. So let me make a step back maybe. So let me recall here the definition of ambiguity. Well, the mathematical details here are not that relevant, although the structure of this formula is important for our proofs. Whatever I'm going to show you is basically proof supported, so there is mathematics out there that supports our claims. Of course, here I'm gonna keep it at a very high level. So we want to find this policy that is free energy minimizing, but yet it is robust. So basically, we want to find now a policy that, a method that finds the optimal policy, minimizing the variation of free energy, but across that ambiguity that Carl was also mentioning just a few seconds before. And how do we do that? So let me frame it first in English. Okay, so again, we want to frame, we want to find the optimal policy. So we are going to find a probability density function or a sequence of probability density function. P star UK given XK minus one. Now, whenever you see the star symbol means optimality, and the curly bracket over there means a sequence, possibly this is a sequence of policies. So we want this solution, this policy to be the solution of a problem where we minimize the variation of free energy. So the optimal policy is the hard mean, is the minimizer of a cost functional that is complexity plus expected cost, just in the way we saw a few, a few minutes before. But there is ambiguity and we must take into account ambiguity. So we are not just minimizing this cost functional, where we are actually accommodating for the worst case. So we are minimizing the maximum free energy, where the maximum is taken across all possible environments, all possible world models that belong to the ambiguity set that I just introduced. So we have a mean max problem where the minimization is as usual in the policy space. But this minimization in the policy space happens on an inner maximization problem, where we maximize over all possible words in the ambiguity set. This is the, let's say, the English translation of our framework.

We can be a little bit more mathematical. Okay, so I'm just just gonna put the problem statement here, which looks, let's say, a huge equation, but it's not that complicated. So again, we want to find the optimal policy, the green, the green part in the sentence. And we want to minimize the variation of free

energy, which is nothing else than complexity and expected costs, where complexity is again complexity between the closed loop system and the generative model. And the expected cost is specified through the, through the action and state costs. And we want to minimize the worst case free energy across ambiguity, which means that we want to minimize the maximum free energy across all possible environments that belong to the ambiguity set I just introduced. So this is basically the formulation, the distributionally robust free energy minimization formulation that we have in the preprint that you see online, that is also linked to the, to the live stream. It is a challenging problem from the mathematical viewpoint, because it is infinite dimensional, and it is a mean max optimization problem.

And it is also a non-linear problem. So it is basically, we cannot rely on linearity of the constraints or linearity of the cost function. Actually, the big step in the preprint online is that this problem can be solved. And this basically gives us the resolution engine. The resolution engine is the set, the software tools and the mathematical methods that make it possible to solve that big optimization problem and make it possible to compute effectively a policy. So how does it work in really a nutshell? The agent, which now we are exploding, so we are doing a little bit more details, gets xk-1, the state, the current state, beliefs or observations, uses the generative model and the state cost. It will compute a policy. And the policy will be computed by minimizing in a recursive way the variation of free energy, the max variation of free energy. So what is the resolution engine? What will it do? And then, also, I will jump in the code. First of all, it can break down at a temporal scale and at a logical level, the original free energy problem. So there are lemmas out there, technicalities, that tell you that the problem can be solved at different time steps and that minimization and maximization have a precise order in which they should be performed. So first we should do maximization and then we should do minimization. So this gives you basically a bi-level approach.

And these software and lemmas, what they do? They yield you an explicit expression for the optimal policy. So we know what the optimal policy is going to be. And also it tells you what is the lowest energy that the agent can achieve under ambiguity. So the optimal value.

From the optimal policy, which is going to be a probability density function like the one that you see in the picture, we are going to sample an action with standard methods. So we are going to sample UK, send it to the environment, and the loop will continue on and on.

So let me start giving you some just the flavor of these key elements here. First of all, the policy can be computed in a recursive way. In particular, it can be shown, I don't do the details here, that the policy can be computed as straight away as a minimization problem. Where the original problem is, this is a perturbation of the original problem, where it will be apparent that to you, that rather than the cost we had before, now we have what we call the cost of ambiguity. We call this cost of ambiguity because it comes from an optimization problem. The maximization step in the environment, in the space that I mentioned before, but this cost is there just because there is ambiguity. If there was no ambiguity, we would not have this step, and then we will recover classic methods, ambiguity-free methods.

So, just two key takeaways here, that there is a free energy maximization step. If you look at the top

diagram, top equation, top formula in green, you will realize that this is maximizing evalational free energy. So, we have a free energy maximization step, and it turns out that this free energy maximization step that gives us a cost of ambiguity is the guy that guarantees robustness. Also, of course, as also Karl was mentioning, the model does not need the agent to know what is the real environment of the real robot model. It only relies on the probability, sorry, on the ambiguity set B eta. Also, in the top formula, there is a C bar there. This comes from a backward recursion. So, one could use dynamic programming or neural approximators, deep neural networks, for example, to estimate these pieces, this piece. I'm not going into the details of this right now.

So, we have two steps, as we said, a logical breakdown of the decision problem. So, we want to find a policy. Finding the policy means minimizing that cost that is now at the center of the slide, in the middle of the slide, which has the cost of ambiguity. And this cost of ambiguity comes from maximizing the free energy. In order to make the model useful, so deployable, essentially, we need to find a way to compute that cost of ambiguity. Cost of ambiguity that, I remind you, comes from this optimization problem. Now, this is a very hard optimization problem, because it is infinite dimensional. So, this is optimization in the space of density. Okay? And it is also a non-linear problem, although in a constraint that is a compact set. It turns out that most of our methods, the formal methods that we developed, were aimed at proving that you can actually compute this cost in a rather efficient way, I would add, or at least with off-the-shelf tools from convex optimization. Okay?

I, this is actually a set of theorems, propositions and lemma, which I'm going to package as a meta-theorem, which is very informally stated. This meta-theorem says that the cost of ambiguity has a specific form, it's eta plus c tilde. Okay? Well, eta is a given. It is the radius of ambiguity and c tilde is what our model computes. Now, this c tilde comes from solving an optimization problem that now is a scalar optimization problem. So, in this formula, alpha is a non-negative scalar value. So, we just have scalar optimization. And b tilde alpha is a function that I'm not going to specify here, but it is convex. So, in the end, we can recast the problem of solving, of finding the cost of ambiguity, and address this problem via a convex scalar optimization problem that, moreover, has a global minimum, of course, because it is convex optimization.

So, we can go back to our resolution engine and say that that general step about finding the maximum free energy can be addressed with a scalar convex optimization problem. So, we did not build a solver for this. We used off-the-shelf tool. So, that the problem that we are going to minimize now, the cost function that we are going to minimize, has a very specific structure. You see that the cost of ambiguity is eta plus c tilde. So, the only thing that is needed to be done is to minimize this guy in the space of policy. Now, this might... Oh, sorry, I forgot. So, what is the engine actually doing? Taking the generative model, it is computing a cost of ambiguity, which sometimes it has this double-shaped potential here. So, this is a cost of ambiguity, means that in minus one and one, the ambiguity is very low, or at least it is lower than other parts. And, for example, towards the end of this diagram, towards the sides of this diagram, the ambiguity is much larger. So, the cost is much, much larger.

So, what does our model do? Computes this cost of ambiguity and will output the policy. And the policy comes from minimizing this cost function here. Now, this might seem as a very abstract, as a rather abstract problem, optimization problem, but actually it is very popular in the literature. And it even has a policy that has a well-defined algebraic solution, which is a softmax. So, what you see here is basically a softmax function, a twisted softmax function. And what is the idea that this formula, that this policy implements? Well, the optimal policy expression is telling us that the optimal solution is given by the generative model, QU, so the bias of the agent, the beliefs of the agent, in a sense. And this model is somehow weighted and or distorted by an exponential. Now, the exponential, and what I think it is quite interesting is that the exponential assigns lower values that are associated to high ambiguity. So, in the end, the probability of getting an action that is very ambiguous is going to be lower and lower, according to our model. Moreover, as I said before, we can also quantify what is the smallest possible free energy, which has a log sum expression, which is standard in the literature from physics, for example. So, there is quite a lot on this slide, so I'd like to emphasize again the optimal policy, the structure. The optimal policy is nothing else than a twisted or distorted version of the generative model, QUK given XK minus 1. And the distortion happens to an exponential function, and the exponential is lower when the ambiguity is large. So, this means that the probability of sampling an action that is associated to high ambiguity is very small. So, the policy in our model has a very clear and neat structure, at least algebraic structure.

What happens inside the model so we can continue developing the flow of information inside the model? So, we have the cost of ambiguity computed with that scalar optimization problem.

We gather the generative model and we build the exponential inside the policy formula.

Then, the two terms here are multiplied with each other, and after normalization, we obtain the policy.

You see, it is quite clear that the policy, the optimal policy coming from multiplication and normalization is a compromise between the generative model and the ambiguity.

So, let me close the loop, going back to the original problem that I was considering, where the classical method would fail in this setting. So, we have our rover, unicycle robot that is moving in this environment filled with obstacles. We have the same generative model as before, the same training and robot model as before, and the same cost. So, exactly the same frame. And I remind you that when we don't take into account ambiguity, bad things can happen in the sense that the robot can crash in the obstacles, unless this is really, unless we really have a trivial situation. With our model instead, with Dr. Free, starting exactly from the same initial conditions, and starting with exactly the same setup, you see that the robot is able to be, to navigate towards the destination. So, for example, take the green, the green trajectory, the green path of the robot, in the right side of the slide, on the right side of the slide, it will crash in the obstacle, in the bottom obstacle. Instead, in the left with our model, in the left part of the slide with our model, it is able to reroute in order to achieve the destination and reach the final destination.

Now, I think I gave quite a few details, and probably the best thing is to jump to the code to see it in action, unless there are questions or comments that you would like to discuss at this stage.

Looks awesome. Please continue.
Then please Josefa, I'll let you share the screen.
Okay.
Please confirm, can you see my screen?
Looks good. Thank you.
Okay. Hello everyone. My name is Josefa, and I'm a postdoc with Giovanni at University of Salerno.
And my background is in control theory. I would like to thank Daniel for inviting us for this presentation.
And I will present the code walkthrough for the robust decision making wire-free energy minimization.
So, we will start with...
We start with the out. First, we'll start with the resolution engine.
Then we'll discuss the robot experiments in more detail. What is the cost structure for this implementation? How we train the dynamics model? How we define the cost function?
What is the ambiguity radius? And what is the cost of ambiguity? How is computed?
And at the end, how are we compute the policy? Basically? And then we'll talk about the in silico and experimental results. And then I will talk about in the end, about the belief of that part, which is a very cool part of this Dr. Free algorithm. Okay, so the resolution engine, as Giovanni just presented a few minutes ago, we want to solve a problem in which we want to obtain optimal policy, a set of optimal policies, which minimizes the maximum free energy cost basically. And we have our model, the real environments model is inside a ball centered around a nominal model p bar. And in the code, I call p bar as a nominal model. And so we decompose this problem into a bi-level problem where we want to minimize over the free energy and the maximization of the free energy is implicitly incorporated in the terms of eta and C tilde, which makes the cost of ambiguity. And we obtain, by solving this problem, we obtain the twisting kernel policy where eta and C tilde are the exponent of our exponential policy. So this is the flow of the Dr. Free Resolution engine. Now, so first I will introduce the robot experiments that we want to use the Robotherium platform by Georgia Tech. And you can send your code to them and they will implement your code on the robots and you can, they will send you the video and the results basically. And what this is a real snapshot of the platform of the work area and the robot, a unicycle robot basically what we want, what our task is that we want to drive the robot successfully to the cross while avoiding the red obstacles basically. The work area is around 3 x 2 meters in size. And there are multiple robots. If you have multi-agent problem also, it's a good platform. But for now, we only consider a single agent here for simplicity. So this is what the code looks like. So first we define the global variables and action space. So we import all the necessary libraries here. The RPS is the Robotherium Python simulator. It's a package by
the Robotherium platform. And we call the Robotherium, which initializes the robots, the transformation from the
unicycle dynamics to the linear dynamics and all the other utilities we call. To solve the optimization problem for

obtaining the cost of ambiguity, we use SciPy. And it's a minimized tool to minimize the function. And we use some other
plotting functions, plotting libraries like Mathplotlib and sklearn to initialize our question process model.
Here I define the action spaces. So action spaces is basically the velocities of the robot in both
axes, x-axis and y-axis. And we define it as a grid of a 5 x 5 discretization. And it ranges from minus 0.5 to 0.5,
the speeds of the robots in both directions. Now, after doing this, first, what we have to do is to load
the dynamics model or train the dynamics model. So the P-bar is the model that we have from the environment
learned by some set of data. And here, in our case, we have three different datasets.
X1, X2, and X3 are the states of the robot and the positions. And the U is the input or the velocity is
given to that robot at a particular time. And we create a stack of these two to make the training input.
And Y is our target. So the next possible state, given the previous state and the current input,
what will be the next state is represented by Y. So we have such three datasets, each consisting of 500
samples. And we train three different Gaussian process models here. And so the Gaussian process,
basically, the most important component of the Gaussian process model is the kernel function. So the
kernel function is basically finding the distance between the two data points at a given time.
So here, the most widely used kernel function in a Gaussian process modeling is a RBF kernel with some
constant term multiplied to it. So here, we define the kernel function, the constant term, the RBF.
The RBF, the radial basis function has parameters called lens scale. And since we have a four dimensional
input, we have four dimensional lens scale parameter, which is a lens scale is a hyperparameter, I'm just
initializing it here, plus some white noise, because white noise, because your data collection channel
can be noisy. And so to model that noise, we have a white kernel function as well here in the entire
kernel function. Now, given this kernel function, we initialize the Gaussian process regressor. And
we take this regressor and we start the fitting and use the fit command basically for this. And we obtain...
So this is the kernel parameter. And what we want to... After training, what we obtain is the hyperparameters
of these kernels. So the sigma f here is the constant kernel coefficient. The RBF here, the radial basis
function here has a parameter w or the lens scale. We have that lens scale here and the noise term here.
So these are the results of the three different data sets for the three different Gaussian models.
Now we take this models and then we move on to defining our cost function for the expected loss here.
So the loss function is basically consisting of three terms. So the first term in the cost is a squared
distance between the goal point and the state. The second term, the cost sum, actually is a RBF kernel,
again, centered around the obstacles. So here are the location of the obstacles on the work area of the
robot theorem. And we calculate the RBF kernels. We center the RBF kernel around these blue points and
we get the cost sum. So basically what it's doing is that when you are near an obstacle, the cost sum,
that second term will have a very high value. So it will shoot up. And the third term, which with a
coefficient five here is for the walls, is for the walls. So we don't want our robot to go outside the
outside the work area. So we don't want to get it outside the rectangle. So basically it's like a

control barrier function. When, if it's near the wall, this term has a very sharp rise and the cost is very high. So this summarizes our cost function. And so the cost function takes the initial state as an input and the goal points where you want to go, the robot wants to go, and the obstacle points as an input and gives out a scalar cost. So this is what, when we compute this state cost, this function for the entire grid, this is what our function looks like. So you can see that you have a very high value when you are far from the goal and you have very high value near the obstacles and around the boundaries. And you have the lowest value near the bottom left where the goal is in our scenario.

So we take this cost function and the Gaussian process model. And now we come to a very important part of the resolution engine is the cost of ambiguity. So the cost of ambiguity as discussed before depends on the two terms, which is eta and C tilde. The eta is the ambiguity radius and the C tilde is the cost, transformed cost, which we define like this. So the C tilde function takes an array of costs from the cost function, the cost function, the cost function, the eta, the ambiguity radius. Here, the p bar is the nominal model probability function. And the generative, I call the generative model, the q in the paper as a reference problem. And it gives out the scalar again in the cost, in the form of transformed cost. So before, Giovanni introduced a term in the C tilde computation, V, V alpha. This objective function is the V alpha. So this is the objective function that we want to minimize basically. And it depends on the parameter alpha. And it consists of two terms. First is the fine term in alpha. So alpha into the ambiguity radius eta. And the term two basically is alpha into the log sum exponential of the cost, which raised to one by alpha. So we want to minimize this objective function over alpha. We take the minimize the minimum value of this objective function. And we basically compare it against the baseline, which is computed by the maximization of this log exp of the cost. And the minimum value gives you C tilde. For more detail, you can refer to the paper, how it is formalized there. So this minimization gives you the C tilde, which is dependent on eta and alpha.

So eta here, we define eta as a KL divergence between the goal points and the next possible state computed from the caution process model. And for computational stability, we clip it between 0 and 100. So KL is basically the distance between, it's not a distance, but it's a difference between the two different distributions, P and Q. And here I made a mistake. I think there should be Q here instead of P. So this is the expression for KL divergence. And for the Gaussian distributions, you can compute the KL divergence in an analytical form, in a closed form, without any sampling.

So now given eta and C tilde, we can compute our policy, which eta and C tilde makes the exponent of our policy.

So the policy here, the P star, the optimal policy is a twisting kernel where the Q is the generative models policy. And the exponential gives you high value if the C tilde, the ambiguity cost is low. In this case, particular case, in this example, we consider Q to be uniform. And since it is uniform, you can basically remove it from this expression of the policy.

So the policy computation code starts, the policy computation code takes the initial state, your control space, because we have two inputs, you have two spaces, the control or action spaces, the goal points and the obstacle points and gives out a, in this case, it will give out a five cross five matrix of a probability,

it's a probability function, from which you can also sample the action that you, that will go to the, to the agent. So for this computation, first we initialize the exponent inside the exponential, with the control space, according to the control space. And we also initialize the, the policy distribution according to control space size. And now we, we go, we compute using the for loops for each possible combination of action. We take the combination of that action. We give this action to the Gaussian process model. And we compute, we predict the next state, the next possible state. And the Gaussian process model will give you basically a mean and a covariance. So the here, the next step nominal is the mean of the Gaussian process and the sigma norm is the, is the covariance. And using this, we compute the samples, the sample from this mean and covariance, the next possible states. And using the sample, we compute the, we compute the cost, sample the costs for this number of samples. So here, okay. So now after, after computing the samples, I compute the eta, the ambiguity radius. Again, the ambiguity radius, as mentioned before, is a scale divergence. And it is clipped between 0 and 100. And I take the number of samples, 100 number of samples, and I compute the cost for all the next possible states. And I give this cost matrix and the eta to the C tilde function. And as I showed earlier, it will give out, give out another scalar value, the transformed cost, which is our, this is our exponent. The minus eta minus C tilde is our exponent. And so here we use the exponential trick metric to, to normalize this and formulate the action PDF, because to avoid any overflow and underflow, we subtract the exponent with the maximum of that exponent matrix. And basically you have the, you have, you have the normalized PDF here. And we, we do the flat flattening operation and unraveling the index. And then we take the sample, the action from that, from this PDF. And we give the action, the, the agent will then give the action to the state, to the system. So given this policy code, now we, we want to apply to the Robotherium example, example, and we perform basically a 12 set of experiments on three different training stages. So the doc, you can see in the top panel on the top is the doctor free algorithm. And on the bottom is the ambiguity and aware panel, where you can see that the, the doctor free agent, even with unknown, unknown system dynamics, it can still reach the goal without colliding any obstacles while the ambiguity and aware agent, although the policy it is computing is also optimal. It, it encounters or collides inside the obstacle. Only in some cases, when you can see that the obstacles are not present in the way of the agent, it will reach the goal. So this is, we, we show that, how our method, uh, basically solves the issue of, uh, the, the model mismatch. And we also have the experimental results for this. So you can see the agent, uh, the robot moving in the environment, the actual robot and how it, uh, avoids the obstacle. So because of some uncertainty, sometimes the robot also, uh, gets a bit confused, let's say, but, but, uh, anyway, it will, it will reach the the destination without, uh, without colliding, which is quite cool.

Okay. Yeah. It's, uh, it gets, okay. Could you just share a little bit, could you just share a little bit right there? What, what is its spatial awareness? Like, is it aware

of its coordinates? Is it have a egocentric or allocentric navigation? Like how does it understand the location and the relative position of the obstacles? So, so, so the, so yeah, okay. Uh, so, uh, here in this case, um, so the, the, the location is provided by the, the camera. So the camera is tracking, the top camera is tracking the robot and it, it updates its location. The camera computes the location and updates the location on the, on the grid. Also it, it, uh, here it, uh, it has the input. It, it knows the obstacles location. So it is pre pre, this is given in the cost function. We define the obstacle points. So it knows the obstacles are, are present. So when it comes near the obstacle, the cost function is automatically very high and it's, it's avoids immediately the obstacle. So, um, I hope, uh, it answers your question. Okay. So it's aware of its two dimensional position and of the obstacle and the goals, and it computes kind of like a suitability that's ambiguity aware. Yeah. Yeah. Yeah. Awesome. Thank you. But, uh, but it's, but, but the thing it does not know is if it takes this action. So if it takes, let's say from the grid, uh, from the action space, it takes, uh, one action. It does not know it, it, it does not know where it will take this, this action will take it. So it cannot foresee accurately where this action will take it. So that's where the ambiguity parts come. So that's where the ambiguity is. So it does not know exactly. Cannot be sure of its own decision.

Oh, I'll just ask one more. Oh yeah. Please go ahead.

Yeah. Okay. No, no, please ask.

Yeah. Well, that's one more question from the live chat. So Andrew Pachea wrote, I'm wondering about what degree of computational overhead, this computing, the worst case ambiguity adds to running the agents. So like just in this situation, what was kind of the computational complexity or the resource use of these different stages of the algorithm? Like does the Dr. Free add a small overhead to, uh, most of the computational cost being on the policy inference or does this ambiguity add a significant computational overhead? Uh, no, uh, it's, uh, for sure because of the com, uh, because of the ambiguity, uh, computation of the ambiguity, uh, cost there is a, there is a bit slower than the, the ambiguity unaware agent, but as you can see, uh, here in the, sorry, in the video, uh, it is all real time. So it is doing that. This is calculating the actions, uh, in real time. Uh, I don't know. The video is gone for some reason. Okay. Ah, yeah. If I, if I, if I may add something, uh, Daniel, I think this is an excellent question because it may be a, a, a research, uh, research program per se. So finding the computational tricks that, uh, improve, uh, improve computational efficiency as much as possible. Yeah. So there are many tricks indeed. Yeah. So the one thing that, uh, would be interesting to see is because I can, if, if you allow me to go back a little bit. Okay. So policy computation here, you can see that we are looping over all the possible actions here, but you don't need to do that. You can have a multi threading or some parallel computation, and you can compute all the actions for, uh, sorry, the computation, the cost for all the possible actions in a parallel computation. And which will, it will, uh, which will, uh, I think make it much more faster because here we are using the for loop over, uh, uh, it's, uh, so it's, it has, uh, if the, if the action space is very large, then it will for sure have a very, uh, high computational overhead, but you can, uh, I think you can, you can resolve it by parallel computation in the, in, for this for loops.

Okay. If, uh, there are no more questions, uh, I will go ahead.

Okay. So, uh, now we'll come to the belief of that part. So belief update, what is belief update is that, um, that, uh, if agent is performing some tasks, we can learn the objective functions of the agent, of the expert agent, just by their data of the input and the states data of that agent. And, uh, uh, uh, the cool part for the, uh, the doctor free policy computation is that the policy is very interpretable. So you can reconstruct the objective function of the agent, uh, simply by using the data, the state and the input data of the expert agent. And we do this by, by solving this, uh, by minimizing the likelihood, negative likelihood, uh, given here. So the, the likelihood function is basically a summation of the affine term plus a log sum exp. And because the, both of the, both of the terms, the affine term and the log sum exp are convex, this optimization problem is convex, even though the, the cost function, uh, or the objective function, uh, of the expert agent is non-convex in nature. So the major, the major components of, for this, uh, belief update are the features, the cost features and the weights, the feature weights, the feature weights is the thing that we want to learn by optimizing, uh, over the, uh, the log likelihood function. And, uh, so, uh, so first, uh, let's see the code structure for this to solve this problem. So again, we define the global variables, the action space, we have to load the train models, uh, for, and the expert agents data. So we have the data of the state and the action for, from some expert, uh, we define the cost features, uh, we compute the feature expectations, uh, using the Gaussian process model. And we formulate and solve this optimization problem here. And we obtain the reconstructed cost.

So, so I will start with the cost features here. What, what is the cost features that we define?

So similar to the previous, uh, task, I am defining the cost feature as the, as the, the three, uh, the two terms. The first is the, the square distance from the goal point.

And the second term is again, the Gaussian, uh, or the RBF kernel centered around some points. And the RBF kernel is defined, uh, by this function here.

Um, so the RBF kernel now is not defined only on the obstacles, the, the actual obstacle in the environment, but now the, the RBF kernel is defined on a set of uniformly distributed points on the grid, which may or may not represent the actual obstacles. So you can see that this is the, the, the feature, the feature map or the feature, uh, grid.

And this is uniformly distributed and the actual, uh, actual obstacles are here and here. So we, that's, so the algorithm has to learn how to, how to give importance to the relevant features only, and not to random features in the, in the environment. Okay. So given this cost features, uh, we solve this minimization problem. Uh, and, uh, so we have, let's say we have M number of samples from the expert agent.

So we initialize the initial state and the, uh, and the, the features. So for, again, for all the possible number of samples, inputs, we compute the next possible state using the, uh, Gaussian process model. And we sample a number of samples from, from this Gaussian process model.

And we compute the features. And, uh, we compute these features, uh, and we basically do the averaging or the, uh, we, we do the, we do the averaging and we compute the expected value for these features.

So here, the first, the second term, uh, of the, uh, of the equation was log sum exponential. And for that, you need to compute the features for all the possible inputs.

While the first term, uh, of the, of the, uh, likelihood function was an affine term, which only computes the given action taken from the expert agent.

So you can see here at the end, there are two terms. The first is the fine term in the W, uh, which, uh, which takes the feature sample, which is dependent only on the input given from the agent.

Uh, while the log sum exponential, uh, is depending on all the possible inputs, uh, the feature computation for all the possible inputs.

So we make this, um, we make this,

And you can see that where there was obstacles in the grid, in the center, you have very high weights, while all the weights are near zero or are very low compared to the real obstacles.

So this was the real cost function or the objective function of the export agent.

And this is the reconstructed cost function of the agent.

And you can see that they are very similar in nature.

So you have high costs away from the goal point and you have the obstacle in the center, so on and so forth.

And using this cost function, the reconstructed cost function, the agent is still able to go to the goal while avoiding the obstacles.

So just to recap, so the cost structure for the robot routing task was to have the define the global variables, action space, load and train the dynamics model.

Cost function and the features, compute the policy and you compute the collected data and basically visualize the robot going to the goal.

While in the belief of that cost structure.

We also defined the goal.

We defined the cost features.

We defined the cost features.

Hello.

Hello.

Yeah.

Sorry.

Go ahead.

It just blipped for a second.

You're back though.

Yeah.

Okay.
Yeah.
Okay.
Okay.
Sorry.
Okay.
Okay.
Okay.
Okay.
Sorry.
Okay.
Okay.
As I was saying, so the belief of that structure.
We also need to define global variables and action space.
We need to load the trained models.
We define the cost features and we compute the feature expectations and we formulate and solve the minimization of the log likelihood function and we obtain the reconstructed cost.
So, yeah.
Okay.
Okay.
If you have any questions now, I'm happy to take them.
Awesome.
Yeah.
I'll ask a few quick questions and if anyone else has a question in the live chat, go for it.
So, first just, it was mentioned that it's infinite dimensional.
So, could you unpack what that means as you're dealing with a finite number of dimensions in terms of sensors and actuators?
So, what does that mean for it to be infinite dimensional?
Okay.
Okay.
Ah, the infinite dimensional was actually here.
So, it's regarding to the, maybe the professor slide is better for this.
I can, maybe, maybe I can put my slides because this point is more clear.
Okay.
Give me just one split second, please.
So, to answer your question, Daniel.
So, do you think that infinite dimensional nature comes from the statement of the problem?

So, the...

So, you are not sharing.

I see it actually.

I see it.

Okay.

So, maybe...

I'm not sure.

It's like on Zoom, like select, maybe you're both sharing or something.

Oh, okay.

But it's good though.

I see your slides.

Okay.

You see it.

Okay.

Perfect.

Okay.

The infinite dimensional that was mentioned before, it's in the mathematics, in the formal

aspect of the framework.

Because this optimization problem is... has as decision variables the functions.

Okay.

So, the infinite dimensional that makes it an infinite dimensional optimization.

When OSEFA engineered Dr. Free on the code, of course you need to estimate this function.

So, he made a step that discretizes at some point some of these functions.

And that of course reduces the problem to the finite dimensional optimization.

So the infinite dimensional feature that we were referring to was about the problem statement.

So the fact that we are optimizing in functional spaces.

Awesome. Okay. Another question, like, how would someone go about adapting this to another setting?

Like, how would they think about framing the kinds of algorithms and approaches that are already in the active inference or the cybernetics literature?

Where do you see opportunities like which systems for applying this and then how adaptable or how would you go about adapting the open source code you've provided to give this sort of ambiguity aware overlay?

So I'll go maybe with the framework and then I'll let Josefa comment more on the code since he developed the code.

So, well, the framework is, I would say, rather general.

So I would say that the system level steps would be to identify a good cost functional, identify the ambiguity set, model the environment, the training model and everything else that comes with it.

Once you have this key and the generative model, of course, once you have these key ingredients, I see these frameworks fitting in the context of sequential policy optimization, which is really sits.

Is that bread and butter of control theory, but also cybernetics.

And it has a lot of interdisciplinary links with signal processing, etc.

In terms of code, I think I will let Josefa answer that.

But the main challenge is going to be the computation of the integrals and the probability density functions that are implied by the framework.

So, I think robotics is one of the fields in sequential decision making, which is very, very cool to apply this because we have seen that when you have mismatch in parameters of the models of the system, the robotics, the task is very difficult to perform.

We also have seen robots fail when they are mismatching the parameters.

So, if somebody wants to apply or implement this framework, I would think that they would first need to change the dynamics model that they have.

So, you need to adapt your dynamics model according to the system that you have, first of all.

Then, for example, if you are working in an environment in which the nominal model and the generative model are not really gaussians, at that point you will not have the analytical solution or the close form solutions as given in the code.

And for that, I think you need to revert to sampling.

Some form of sampling to compute the integrals and the expectations.

So, that's one part that I think if someone wants to implement this in a system which is more complicated and doesn't have gaussians, assumptions, then you need to consider sampling.

Another part would be, as I said, if the action space of the system is too large, then to avoid operational overhead, you need to consider parallel computing, maybe, or some form of vectorization to avoid the for loops in the input.

Yeah.

That's, I think that's the core part.

But other things like the cost of ambiguity, the function that computes this cost of ambiguity, the structure would be the same for that.

Yeah.

Awesome.

So, sort of in closing, what are your next steps for research or how do you see this continuing or what ways would you like to see people in the community kind of continue with the work that you're doing?

Well, first of all, I'd say that if you want to use your code and you have any questions, I think we would be happy to answer these questions.

And we are happy if the code is useful beyond, of course, the use case we are considering.

Robotics being one remarkable example, I think.

And where do we see this place?

I think we prepared this slide that summarizes and maybe captures some of the points that were discussed and about the interdisciplinary link.

So we see these place in the context of feedback systems.

So in the end, the agent and the environment are two systems that are in feedback.

So these reactions affect state and state affect the actions.

And and basically what we did was to bring some control theoretical and optimization methods into this

minimization of variation of free energy.

So it is really an intersection between competition embodiment of agents and robustness possible.

I think very cool research ideas could be integrating deep reinforcement learning.

So the techniques that we that technique that we presented, Dr.

Free is not mutually excluding with deep reinforcement learning.

One could use deep reinforcement learning to learn ambiguity.

One could use deep reinforcement learning to learn a better cost, etc.

So I think there is a sweet spot of applications that can have a high impact for real world deployments by merging these two approaches.

There are also there is also another.

I'm not a neuroscientist.

I'm going to start with that.

But I think the model has some cognitive implications.

As I've just showed, it supports Bayesian belief updates.

But one thing that we had no time to discuss about, of course, is that something very nice happens in the regimes of large and small ambiguities.

So basically when ambiguity is small, you get the behavior of an agent that knows everything about its model.

And it turns out that nobody cannot perform this agent.

So it's the best possible agent.

It is an Oracle essentially that does the best possible actions, will achieve the best cost function.

On the other side of the spectrum, instead, when ambiguity is very high, there is a sort of a transition where we can show it is shown in the in the preprint that the optimal policy is training independence in the sense that is independent on the training model.

So it is a kind of situation where the agent dominated by ambiguity grounds its decisions just on ambiguity and not on what it learned previously, which is I think it's a kind of cool behavior.

Of course, I'm not a neuroscientist, but I think it's something that could be investigated on that side, on that spectrum, on that side of the spectrum.

I hope I answered your question.

Yeah, that's an interesting point, kind of like neuroplasticity and age and how you never do worse by knowing more about the environment, all things being equal.

And then as you turn up the dial on ambiguity, it erases whatever understanding was made ambiguous.

Yeah.

And that could be even empirically fit to understand, like, what are the biological contexts that different ambiguities, different hyperparameters are in play.

Yeah, yeah, exactly.

Yeah.

Well, thank you both and to Carl for joining.

We really appreciate the work and for sharing it and having the open source code.

It's really cool.

Thank you.

I put here two links to the papers in the code, QR codes in case you want to, you're curious and you want to check them out.

And thank you again, Daniel for organizing this.

Awesome.

Okay.

Till next time.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.